

ENTREGÁVEL 2 – DOCUMENTO ARQUITETURAL

Sistema de Controle de Entrada e Saída – Loja de Simulador de Aviação

1. Contexto e Objetivo do Documento

Este documento apresenta a evolução arquitetural do Sistema de Controle de Entrada e Saída para uma Loja de Simulador de Aviação. O sistema foi concebido para solucionar problemas relacionados ao gerenciamento de estoque, controle de entradas e saídas de produtos, rastreamento de movimentações e geração de relatórios gerenciais. Como evolução da primeira etapa do projeto, foram aplicados cinco padrões de projeto GoF: Observer, Proxy, Factory Method, Mediator e Visitor. O objetivo principal é melhorar a modularidade, reduzir acoplamento, facilitar manutenção, aumentar a reutilização de código e preparar o sistema para futuras expansões.

2. Visão Geral da Arquitetura

A arquitetura é composta pelos módulos Estoque, Movimentação, Histórico de Movimentações, Usuário, Validação de Operações e Relatórios. Os componentes trabalham em conjunto para controlar o fluxo de entrada e saída dos produtos comercializados pela loja. A solução foi estruturada utilizando princípios de orientação a objetos e padrões de projeto para garantir organização e escalabilidade.

3. Observer (Comportamental) – Notificações de Estoque

Problema identificado: Funcionários e administradores precisam ser informados sempre que ocorrerem alterações críticas no estoque, como níveis mínimos de produtos ou reposições importantes.

Solução proposta: Aplicação do padrão Observer utilizando EstoqueSubject como Subject e NotifFuncionario e NotifAdministrador como Observers.

Justificativa técnica: O padrão permite comunicação um-para-muitos com baixo acoplamento, evitando dependências diretas entre estoque e notificadores.

Impacto arquitetural: O sistema passa a suportar novos canais de notificação sem alterar o núcleo do estoque.

Vantagens: extensibilidade, baixo acoplamento, atualização automática e facilidade de manutenção.

Limitações: excesso de notificações e maior dificuldade para rastrear fluxos orientados a eventos.

4. Proxy (Estrutural) – Controle de Acesso ao Estoque

Problema identificado: Nem todos os usuários devem possuir acesso irrestrito às operações do estoque.

Solução proposta: Utilização de ProxyEstoque para intermediar chamadas ao EstoqueService.

Justificativa técnica: Centraliza verificações de autorização e validação antes da execução das operações.

Impacto arquitetural: Os módulos passam a depender da interface IEstoqueService em vez da implementação concreta.

Vantagens: segurança, auditoria, controle e encapsulamento.

Limitações: aumento de complexidade e inclusão de uma camada adicional.

5. Factory Method (Criacional) – Criação de Produtos

Problema identificado: A loja trabalha com múltiplos tipos de produtos, exigindo diferentes processos de criação.

Solução proposta: Aplicação do padrão Factory Method através da classe ProdutoFactory.

Justificativa técnica: Centraliza a criação de objetos e reduz dependência entre módulos e classes concretas.

Impacto arquitetural: Facilita inclusão de novos tipos de produtos sem alterar código existente.

Vantagens: reutilização, flexibilidade e escalabilidade.

Limitações: aumento da quantidade de classes.

6. Mediator (Comportamental) – Coordenação dos Módulos

Problema identificado: Os módulos de entrada e saída dependem fortemente um do outro.

Solução proposta: Introdução do EstoqueMediator para coordenar a comunicação.

Justificativa técnica: Reduz dependências diretas entre componentes.

Impacto arquitetural: Comunicação centralizada e desacoplada.

Vantagens: manutenção facilitada, organização e modularidade.

Limitações: risco de concentração excessiva de responsabilidades no mediador.

7. Visitor (Comportamental) – Geração de Relatórios

Problema identificado: Necessidade de gerar diferentes relatórios sem alterar classes dos produtos.

Solução proposta: Implementação de Visitors especializados para relatórios de inventário e movimentação.

Justificativa técnica: Permite adicionar novas operações sem modificar a estrutura dos elementos visitados.

Impacto arquitetural: Separação entre estrutura de dados e operações de relatório.

Vantagens: flexibilidade e facilidade de expansão.

Limitações: maior quantidade de interfaces e classes.

8. Trecho Consolidado da Arquitetura

Usuário → ProxyEstoque → EstoqueService → Estoque EstoqueSubject → NotifFuncionario /
NotifAdministrador ProdutoFactory → PecaSimulador / SoftwareSimulador / AcessorioVoo
ModuloEntrada ↔ EstoqueMediator ↔ ModuloSaida Visitor → RelatorioInventario /
RelatorioMovimentacao

9. Considerações Finais

A aplicação dos padrões GoF permitiu transformar o sistema em uma solução mais robusta, modular e preparada para crescimento. Observer automatiza notificações, Proxy protege operações críticas, Factory Method organiza a criação de produtos, Mediator coordena módulos independentes e Visitor amplia a geração de relatórios. O resultado é uma arquitetura alinhada às boas práticas de Engenharia de Software, atendendo aos requisitos acadêmicos do projeto e fornecendo uma base sólida para futuras evoluções.